Chapitre 05
Exploration modale et comparaison de modes

Du degré caractéristique au manche comparatif, avec les premiers tests unitaires Vitest.

Mi mineur naturel



Mi dorien



Séance : 5 **Date du chapitre :** 25 octobre 2026 **Projet :** React + TypeScript strict + Vite + pnpm

Périmètre : musique, logique métier, tests unitaires, affichage du manche, comparaison, légende. Pas d'audio, de tablature ni de backend.

Portée documentaire. Ce chapitre retrace les décisions et les extraits réellement échangés pendant la séance. Les exemples complets indiqués « reconstruction de référence » sont des synthèses pédagogiques, à confronter au dépôt pour reproduire les détails exacts.

Comparaison de modes



Sommaire et contrat de lecture

01. Point de départ et objectifs de la séance
02. Théorie musicale et degré caractéristique
03. Modèle TypeScript et calcul réutilisable
04. Introduction de Vitest et stratégie de tests
05. Intégration React et lecture du manche
06. Comparaison de modes : logique métier et tests
07. Comparaison : état, sélecteur et affichage
08. CSS : priorités et double coloration
09. Composant de légende
10. Difficultés rencontrées et décisions
11. Reconstitution pas à pas et validation
12. Prochain incrément et prompt de reprise
Annexe : extraits de code de référence

Convention de preuve utilisée dans le document

Mention	Ce qu’elle signifie
Confirmé en séance	Le code a été montré, ou le fonctionnement / les tests ont été déclarés validés.
Reconstruction de référence	Code synthétique proposé pour documenter l’intention ; pas une copie certifiée du dernier fichier local.
À revérifier	Commande ou détail du dépôt non communiqué dans la conversation.

Le document de démarrage du projet fixe le principe d’un apprentissage ludique, local et incrémental. Le manche et les futurs éléments de composition doivent partager des données cohérentes ; cette séance conserve le périmètre sans audio ni tablature.

01 / Point de départ et objectifs

À l’ouverture de la séance 5, les quatre premières séances avaient déjà produit un manche interactif de six cordes, cases 0 à 12, avec calcul des hauteurs MIDI, sélection de tonique, affichage notes / intervalles et coloration des notes de gamme, toniques et note sélectionnée.

Élément existant	Rôle au début de la séance
SCALES	Quatre gammes : minor, major, dorian, phrygian ; intervalles en demi-tons.
getIntervalSemitones()	Distance chromatique modulo 12 entre une tonique et une note MIDI.
getIntervalLabel()	Correspondance d’une distance avec son étiquette d’intervalle.
isNoteInScale()	Détermine l’appartenance d’une hauteur à la gamme active.
displayMode	Choix des étiquettes de notes ou de degrés.

Élément existant	Rôle au début de la séance
selectedPosition	Position cliquée : corde et frette, ou null.

Objectif initial : distinguer sur le manche les degrés choisis comme repères modaux. **Objectif enrichi en séance :** installer des tests unitaires, comparer deux modes de même tonique, puis afficher une légende contextuelle.

Principe maintenu : une seule tâche à la fois ; la logique musicale reste indépendante de React ; l'utilisateur écrit et vérifie son code.

02 / Musique : couleurs modales et comparaisons

La sixte du dorien est le sixième degré (**index 5** dans un tableau de sept notes), mais sa distance à la tonique est de **9 demi-tons**. Ces deux nombres décrivent des concepts distincts : position dans une gamme contre distance chromatique.

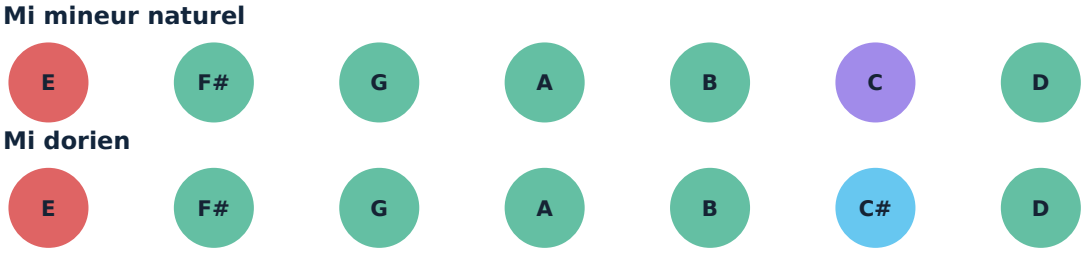
Mode en Mi	Notes	Repère(s) discuté(s)	Demi-tons
Dorien	E F# G A B C# D	6 (C#)	9
Phrygien	E F G A B C D	♭2 (F)	1
Mineur naturel	E F# G A B C D	♭6 (C) ; ♭7 (D) évoquée	8 ; 10
Majeur	E F# G# A B C# D#	3 (G#), 7 (D#)	4, 11

La septième mineure du mineur naturel a été proposée dans la discussion. Son ajout définitif au tableau n'a pas été montré explicitement ; il faut vérifier sa présence dans le fichier local. Les degrés repères **ne sont pas nécessairement exclusifs** au mode : par exemple ♭7 est aussi présente en dorien et en phrygien.

La quarte du majeur est une **quarte juste**, commune aux quatre gammes ici présentes ; elle ne discrimine pas leurs couleurs. Le majeur est donc mis en évidence par sa tierce majeure (4 demi-tons) et sa septième majeure (11).

Différence entre repère caractéristique et différence effective

En exploration simple, on choisit des degrés musicalement instructifs pour un mode. En comparaison, on mesure une différence **effective entre deux ensembles de notes**. Ainsi, C et C# distinguent Mi mineur naturel et Mi dorien ; leurs six autres notes sont communes.



Cette distinction commande les deux stratégies d'affichage : exploration = couleurs des degrés ; comparaison = notes communes / propres à chaque mode.

03 / Données TypeScript : déclarer plutôt que multiplier les if

La première solution, conditionner chaque note à `scaleName === "dorian"` et à un intervalle de 9 demi-tons, fonctionne mais ne passe pas à l'échelle. Le choix retenu est de conserver SCALES inchangé et d'ajouter une structure séparée, par souci de simplicité et de rétrocompatibilité.

```
export const CHARACTERISTIC_INTERVALS:
  Partial<Record<ScaleName, number[]>> = {
    dorian: [9],
    phrygian: [1],
    major: [4, 11],
    minor: [8], // verifier si 10 a ete ajoute dans le depot
  };
```

Pourquoi `Partial<Record<ScaleName, number[]>>` **?** `Record` limite les clés aux gammes déclarées ; `Partial` permet d'introduire progressivement les modes. La lecture d'une propriété peut donc retourner `undefined`.

Détection générique

```
export function isCharacteristicInterval(
  tonicMidi: number,
  noteMidi: number,
  scaleName: ScaleName
): boolean {
  const distance = getIntervalSemitones(tonicMidi, noteMidi);
  return CHARACTERISTIC_INTERVALS[scaleName]
    ?.includes(distance) ?? false;
}
```

Reconstruction de référence. Cette version factorise les contrôles MIDI déjà effectués par `getIntervalSemitones()`, supprime la répétition de l'accès au dictionnaire et combine `?.` et `?? false` pour retourner un booléen même si le mode n'est pas configuré.

Garde-fou : éviter de dupliquer la validation MIDI quand une fonction appelée en est déjà responsable. En revanche, `isNoteInScale()`, tel que montré, ne valide pas ses paramètres ; la fonction de comparaison a donc conservé ces contrôles explicites.

04 / Introduction de Vitest

Au lieu d'un `console.assert()` temporaire, la séance a introduit une suite de tests durable. Vitest a été installé comme dépendance de développement dans un projet Vite / pnpm. Aucun environnement DOM ni bibliothèque de test React n'a été introduit : les fonctions musicales sont des fonctions TypeScript pures.

```
pnpm add -D vitest
pnpm exec vitest --version
```

Ajout du script au `package.json` :

```
"scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "lint": "eslint .",
  "preview": "vite preview",
  "test": "vitest run"
}
```

`vitest run` effectue une exécution unique, adaptée à une future intégration continue. L'option `vitest` seule lancerait le mode watch.

Premier test et cas limites

```
import { describe, it, expect } from "vitest";
import { isCharacteristicInterval } from "./scales";

describe("isCharacteristicInterval", () => {
  it("finds the major sixth in E dorian", () => {
    expect(isCharacteristicInterval(40, 49, "dorian"))
      .toBe(true);
  });
  it("rejects the minor sixth in E dorian", () => {
    expect(isCharacteristicInterval(40, 48, "dorian"))
      .toBe(false);
  });
});
```

Le premier cas utilisait bien les valeurs MIDI **40 (E2)** et **49 (C#3)**, avec 9 demi-tons d'écart. Un test négatif a été validé : sans lui, une fonction retournant systématiquement `true` réussirait le cas positif.

TDD : Red → Green → Refactor

Pour le phrygien, le test sur E2 (40) → F2 (41) et le résultat `true` a d'abord échoué ; l'ajout de `phrygian: [1]` l'a fait réussir. Aucun refactor de la fonction n'a été nécessaire : enrichir les données suffisait. Un cycle analogue a servi à enrichir le majeur (D#3, MIDI 51 ; G#2, MIDI 44).

Test / scénario	Attendu	Observation en séance
E2 → C#3, dorian (40 → 49)	true	Réussi
E2 → C3, dorian (40 → 48)	false	Réussi
Mode non configuré à ce moment, phrygian	false	Réussi avant ajout de phrygian
E2 → F2, phrygian (40 → 41)	true	Red puis Green déclarés
E2 → C3, minor (40 → 48)	true	Green déclaré
E2 → D#3, major (40 → 51)	true	Validé
E2 → G#2, major (40 → 44)	true	Validé

05 / React : une information calculée à chaque rendu

Le manche est rendu à partir de six objets `GuitarString` (notes à vide MIDI 64, 59, 55, 50, 45, 40) et de treize frettes (0 à 12). À chaque cellule sont déjà calculés la sélection, la hauteur MIDI, son étiquette, l'appartenance à la gamme et la tonique.

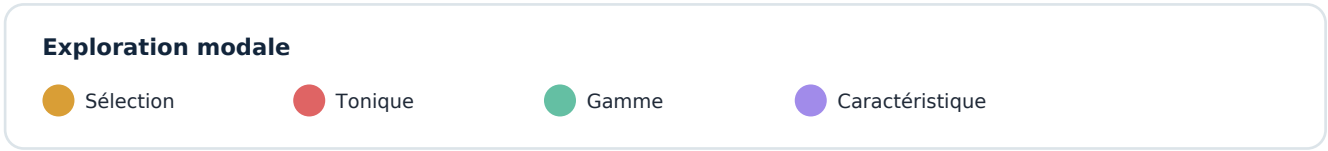
```
const isSelected = cellIsSelected(
  string.pos, fret, selectedPosition
);
const midiNote = getMidiNote(string.openMidi, fret);
const isInScale = isNoteInScale(midiNote, tonic, scaleName);
const isTonic = midiNote % 12 === tonic % 12;
const isCharacteristic = isCharacteristicInterval(
  tonic, midiNote, scaleName
);
```

Le nom `isCharacteristic` est une **valeur dérivée** : aucun `useState()` ni `useEffect()` supplémentaire n'est nécessaire. Quand `setScaleName()` change la gamme, React réexécute le composant, calcule son arbre d'interface, applique si besoin les différences au DOM (commit), puis le navigateur peut repeindre l'affichage (paint).

Premier affichage des degrés

Une classe conditionnelle `is-characteristic` a été ajoutée à chaque cellule correspondante. Les degrés de gamme sont visibles sur l'ensemble du manche et la note sélectionnée reste visible même en dehors de la gamme.

La représentation évolue ensuite vers une **double coloration** : fond de case + pastille portant la note ou l'intervalle. Ce choix a été demandé et confirmé, car le fond aide à lire les motifs géométriques du manche.



06 / Comparer deux modes : logique métier et tests

Pour comparer des modes, les deux gammes partagent la même tonique. La fonction classe une note dans l'une de quatre catégories : commune, seulement principale, seulement comparée ou extérieure aux deux. La comparaison est indépendante de React.

Statut retourné	Interprétation
common	Présente dans les deux gammes
main-only	Uniquement dans la gamme principale
comparison-only	Uniquement dans la gamme comparée
outside	Absente des deux gammes

Les premiers noms `primaryScale` / `referenceScale` ont été jugés ambigus : « référence » pouvait désigner la gamme de départ. Ils ont été remplacés par `mainScale` / `comparisonScale`, avec renommage corrélatif des statuts. Ce refactor de vocabulaire a été validé par les tests.

```
export type ScaleComparisonStatus =
  | "common"
  | "main-only"
  | "comparison-only"
  | "outside";

export function getScaleComparisonStatus(
  tonicMidi: number,
  noteMidi: number,
  mainScale: ScaleName,
  comparisonScale: ScaleName
): ScaleComparisonStatus {
  checkMidiRange(noteMidi);
  checkMidiRange(tonicMidi);
  const main = isNoteInScale(noteMidi, tonicMidi, mainScale);
  const other = isNoteInScale(
    noteMidi, tonicMidi, comparisonScale
  );
  if (!other && !main) return "outside";
  if (other && !main) return "comparison-only";
  if (!other && main) return "main-only";
  return "common";
}
```

Reconstruction de référence. La logique conditionnelle ci-dessus correspond à celle montrée par l'utilisateur ; seuls les noms ont été adaptés après le refactor validé. Les quatre combinaisons booléennes sont exhaustives.

Les quatre tests de comparaison

```
describe("getScaleComparisonStatus", () => {
  it("common", () =>
    expect(getScaleComparisonStatus(40, 52, "minor", "dorian"))
      .toBe("common"));
  it("main only", () =>
    expect(getScaleComparisonStatus(40, 48, "minor", "dorian"))
      .toBe("main-only"));
  it("comparison only", () =>
    expect(getScaleComparisonStatus(40, 49, "minor", "dorian"))
      .toBe("comparison-only"));
  it("outside", () =>
    expect(getScaleComparisonStatus(40, 53, "minor", "dorian"))
      .toBe("outside"));
});
```

Les quatre cas ont été déclarés verts, puis les assertions affectées ont été mises à jour après le renommage.

07 / Comparaison : état, sélecteur et rendu

Un seul état suffit : `comparisonScale: ScaleName | null`. Sa valeur `null` signifie que la comparaison est inactive ; il est inutile de lui associer un second booléen.

```
const [comparisonScale, setComparisonScale] =
  useState<ScaleName | null>(null);

const handleComparisonScaleChange = (
  { target }: React.ChangeEvent<HTMLSelectElement>
) => {
  if (target.value === "") {
    setComparisonScale(null);
    return;
  }
  setComparisonScale(target.value as ScaleName);
};
```

Deux difficultés concrètes ont été corrigées : `value={""}` maintenait le sélecteur en « Aucune comparaison », et l'absence de `return` après `setComparisonScale(null)` déclenchait ensuite une seconde mise à jour vers la chaîne vide. Un as `ScaleName` n'effectue pas de validation à l'exécution.

```
<select
  id="comparison-select"
  value={comparisonScale ?? ""}
  onChange={handleComparisonScaleChange}
>
  <option value="">Aucune comparaison</option>
  {Object.keys(SCALES).map(scale => (
    <option key={scale} value={scale}>{scale}</option>
  ))}
</select>
```

Calculer un statut sans nouvel état

```
const comparisonStatus = comparisonScale
  ? getScaleComparisonStatus(
      tonic, midiNote, scaleName, comparisonScale
    )
  : null;

const note =
  isSelected || isInScale ||
  comparisonStatus === "comparison-only"
    ? label
    : "";
```

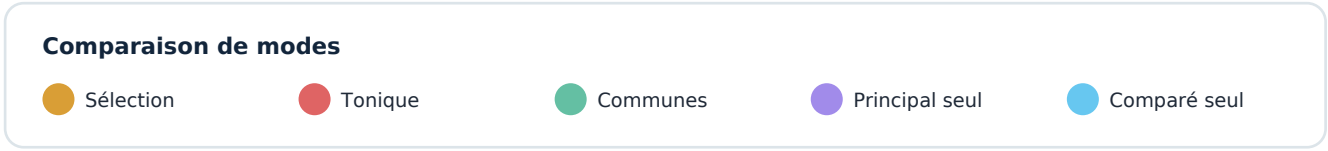
Cette condition d'étiquette est essentielle : une note présente uniquement dans le mode comparé est, par définition, hors du mode principal ; sans ce changement elle resterait masquée. Les classes `main-only` et `comparison-only` sont ensuite ajoutées conditionnellement.

```
className={`fret-cell
  ${isSelected ? "is-selected" : ""}
  ${isInScale ? "in-scale" : ""}
  ${isTonic ? "is-tonic" : ""}
  ${comparisonScale === null && isCharacteristic
    ? "is-characteristic" : ""}
  ${comparisonStatus === "main-only" ? "main-only" : ""}
  ${comparisonStatus === "comparison-only"
    ? "comparison-only" : ""}
`}
```

La classe caractéristique est volontairement désactivée pendant la comparaison pour ne pas superposer deux sémantiques différentes (repère modal / différence entre ensembles).

08 / CSS : double coloration et priorité visuelle

Le premier affichage colorait seulement le fond des cases et réservait la pastille `::after` à la sélection. La séance a étendu cette pastille aux notes de gamme, puis aux notes propres au mode comparé, tout en conservant les fonds colorés à la demande de l'utilisateur.



Priorité	Couleur / rôle	Exploration	Comparaison
1	Orange : sélection	Sélection	Sélection
2	Rouge : tonique	Tonique	Tonique
3	Violet	Degré repère	Principale seule
4	Bleu clair	Non utilisé	Comparée seule
5	Vert	Note de gamme	Note commune

Les règles de pastille partagent leur géométrie et leur contenu via `content: attr(data-note)`. Les classes spécialisées changent ensuite `background-color` sur le fond et sur `::after`. L'ordre des règles CSS, à spécificité égale, assure les priorités ; l'ordre des classes dans `className` ne les définit pas.

```
/* Extrait de référence : compléter par le CSS existant. */
.is-selected::after,
.in-scale::after,
.comparison-only::after {
  content: attr(data-note);
  /* position, diametre, typo, cercle... */
}
.in-scale { background: var(--scale-color); }
.in-scale::after { background: var(--scale-color); }
.main-only,
.main-only::after { background: var(--characteristic-color); }
.comparison-only,
.comparison-only::after { background: var(--comparison-color); }
.is-characteristic,
.is-characteristic::after {
  background: var(--characteristic-color);
}
.is-tonic,
.is-tonic::after { background: var(--tonic-color); }
.is-selected::after { background: var(--selected-color); }
```

Attention : les valeurs exactes des variables CSS et leur emplacement de déclaration n'ont pas été montrés après toutes les modifications. Vérifier dans le dépôt que les couleurs des fonds, des pastilles et de la légende coïncident. Le vert vu dans le code de légende initial était `#005e41`, tandis que la palette proposée utilisait `#64bfa3`.

09 / FretboardLegend : une légende contextuelle

Un composant de présentation dédié a été introduit sous le manche. Sa seule prop utile est `isComparing: boolean`. Sa première version changeait le titre (« Exploration modale » / « Comparaison modale »). Le typage a été simplifié pour décrire **l'objet de props**, et non un booléen isolé.

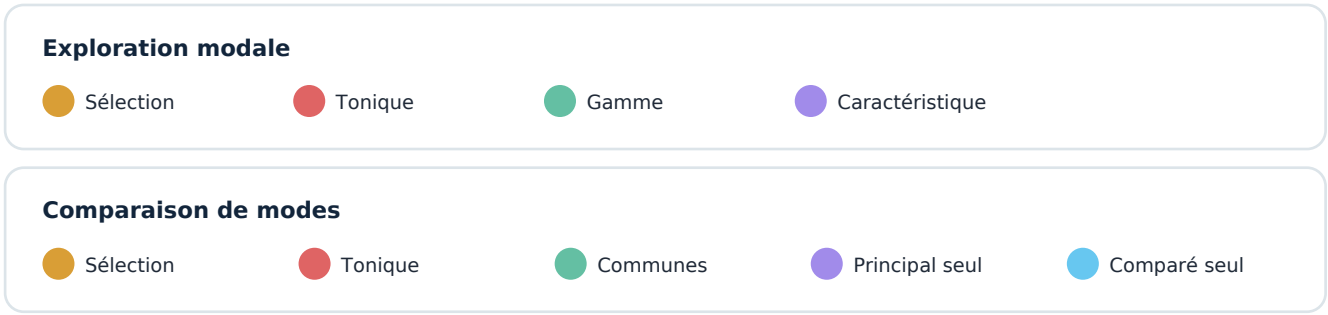
```
type FretboardLegendProps = {
  isComparing: boolean;
};

export function FretboardLegend(
  { isComparing }: FretboardLegendProps
) {
  // rendu derive des props
}
```

Plutôt que deux tableaux de légendes dupliqués, un seul tableau `legendItems` adapte les libellés vert/violet au contexte et ajoute conditionnellement le bleu avec le spread `...(condition ? [element] : [])`. Cela évite un élément `null` dans le tableau.

```
const legendItems = [
  { label: "Note sélectionnée", color: "var(--selected-color)" },
  { label: "Tonique", color: "var(--tonic-color)" },
  {
    label: isComparing
      ? "Mode principal uniquement" : "Degré caractéristique",
    color: "var(--characteristic-color)"
  },
  {
    label: isComparing ? "Note commune" : "Note de la gamme",
    color: "var(--scale-color)"
  },
  ...(isComparing
    ? [{ label: "Mode comparé uniquement",
         color: "var(--comparison-color)" }]
    : [])
];
```

Le JSX a été déclaré terminé par l'utilisateur : les éléments sont produits par `map()`, avec key, texte dérivé de `label` et couleur passée par `style={{ backgroundColor: item.color }}`. Mise en page retenue : flex horizontal, `flex-wrap`, espacement, pastilles rondes de 16 px.



10 / Difficultés rencontrées et décisions

Problème observé	Décision / enseignement
Condition if par mode trop verbeuse	Dictionnaire caractéristique séparé, extensible sans dupliquer les conditions.
Index 5 vs 9 demi-tons	Séparer rang dans la gamme et distance chromatique.
Undefined avec Partial	Optional chaining (?.) et fallback (?? false).
Contrôles MIDI redondants	Ne pas revérifier dans la détection si le calcul appelé s'en charge.
referenceScale sémantiquement ambigu	Renommer mainScale / comparisonScale et adapter les tests.
Sélecteur React bloqué sur chaîne vide	Lier value à l'état via comparisonScale ?? "".
Deux setState pour valeur vide	Un return après le cas null, ou un seul appel conditionnel.
C# masqué dans comparaison	Afficher aussi les notes avec statut comparison-only.
Classes CSS concurrentes	Séparer exploration / comparaison ; ordonner la cascade.
Perte du repérage visuel du manche	Conserver fond coloré et pastille ensemble.
Légende répétitive et null dans tableau	Un tableau avec libellés conditionnels et spread conditionnel.

Les décisions ci-dessus privilégient la compréhension et des incréments simples. Aucun contexte global, mémoïsation, composant générique de design system ou abstraction supplémentaire n'a été introduit.

11 / Reconstituer l'incrément depuis la fin de la séance 4

Pour reconstruire intégralement l'application depuis zéro, il faudra combiner ce chapitre aux chapitres 1 à 4 (manche, modèles de notes, gammes, affichage des intervalles). À partir d'un état équivalent à celui décrit en ouverture de ce chapitre, suivre cet ordre :

1. Vérifier la base. React / TypeScript strict / Vite / pnpm ; scales.ts, notes.ts et manche opérationnels ; quatre gammes et affichage notes / intervalles.

2. Créer les données de repères. Ajouter CHARACTERISTIC_INTERVALS dans scales.ts en utilisant Partial> ; configurer dorian, phrygian, major et minor selon les tests validés.

3. Créer le détecteur. Implémenter isCharacteristicInterval() avec getIntervalSemitones(), includes(), ?. et ?? false.

4. Introduire Vitest. pnpm add -D vitest ; script test : vitest run ; scales.test.ts près de scales.ts ; tests positifs/négatifs.

5. Colorer les degrés. Calculer isCharacteristic par case ; ajouter la classe is-characteristic ; adopter les pastilles ::after tout en conservant la couleur de fond.

6. Créer la comparaison pure. Déclarer ScaleComparisonStatus et getScaleComparisonStatus() ; couvrir les quatre statuts par des tests.

7. Ajouter un sélecteur contrôlé. State comparisonScale: ScaleName | null ; option vide ; value={comparisonScale ?? ""} ; onChange revenant à null.

8. Faire apparaître les notes comparées. Calculer comparisonStatus ; étendre la condition note ; ajouter les classes main-only et comparison-only.

9. Distinguer les deux stratégies. Ne poser is-characteristic que sans comparaison ; définir la priorité CSS et vérifier le sens inverse de la comparaison.

10. Créer la légende. Composant FretboardLegend ; prop isComparing ; tableau legendItems ; map ; CSS flex avec retour à la ligne.

Contrôles manuels recommandés

Configuration	Résultat attendu
Mi dorien, exploration	C# violet ; E rouge ; autres notes de gamme vertes.
Mi phrygien, exploration	F violet (♭2).
Mi majeur, exploration	G# et D# violets (3 et 7).
Mi mineur / Mi dorien	C principal seul violet ; C# comparé seul bleu ; six notes communes vertes sauf tonique rouge.
Mi dorien / Mi mineur	C# principal seul violet ; C comparé seul bleu.
Clic sur note comparée	Pastille orange sur fond bleu ; information sélectionnée conservée.
Aucune comparaison	Retour de la légende à quatre éléments et des couleurs caractéristiques.

Commandes de validation en fin de séance

```
pnpm test
pnpm lint
pnpm build

git status --short
```

État connu : les tests unitaires développés et les comportements visuels ont été déclarés réussis par l'utilisateur. **À revérifier :** aucun résultat terminal final de `pnpm lint` ou `pnpm build` n'a été partagé à la clôture. La réalisation du commit final n'est pas attestée ici.

12 / Bilan pédagogique et reprise de la séance 6

Axe	Acquis pratiqués pendant la séance
Musique	Degrés chromatiques, index de gamme, différences de modes, tierce / septième majeures, ♭2 et ♭6.
TypeScript	keyof, as const déjà présents ; Partial> ; union littérale ; null / undefined ; includes.
React	Valeur dérivée, contrôle d'un select, conversion string vers null, props, map, key ; render / commit / paint.
CSS	Pseudo-élément ::after, data-note, variables, cascade, priorités, double coloration, flex-wrap.
Qualité	Vitest, cas positifs/négatifs, Red-Green-Refactor, refactor de noms sécurisé par tests.

Décision ouverte pour le prochain incrément

Ne pas lancer automatiquement audio, tablature ou backend. Une bonne séance 6 serait de consolider la comparaison modale : vérifier le typage du sélecteur, rendre les différences plus explicites (par exemple des libellés C → C#), ajouter quelques tests limites et choisir ensuite le prochain jeu musical. La décision appartient à la prochaine séance.

Prompt de reprise prêt à copier

« Nous reprenons **Modal Guitar Lab — Séance 6**. La séance 5 a introduit Vitest, les degrés caractéristiques (dorian, phrygian, minor, major), la double coloration des cases et pastilles, une fonction `getScaleComparisonStatus()` testée sur quatre statuts, un sélecteur de mode comparé (`ScaleName | null`), les classes `main-only` / `comparison-only`, et `FretboardLegend` adaptatif. Le mode exploration utilise les repères caractéristiques ; le mode comparaison montre les notes communes et les différences avec une tonique commune. Vérifier d'abord `pnpm test`, `pnpm lint`, `pnpm build`, puis confirmer le contenu exact de `CHARACTERISTIC_INTERVALS.minor` (♭7 discutée). Travaillons une seule fonctionnalité à la fois, en me laissant écrire le code, avec explications TypeScript / React / musique, tests ciblés, puis un chapitre PDF de documentation en fin de séance. Pas d'audio ni de tablature tant que nous ne l'avons pas décidé. »

Annexe / Références et limites de la reconstruction

Source principale : échanges complets de la séance 5, incluant le fichier `scales.ts` partagé, les extraits JSX/CSS, les assertions Vitest et les validations successives de l'utilisateur.

Document de cadrage : `modal-guitar-lab-demarrage.md`, fourni dans le projet : architecture locale Vite + React + TypeScript, apprentissage par petites fonctionnalités, séparation logique métier / affichage et caractère facultatif du projet.

Non inclus : fichiers complets et dernière arborescence du dépôt, configuration CSS finale exacte, rapport de sortie complet de Vitest, rapports ESLint et TypeScript. Des fragments pédagogiques ne doivent pas être interprétés comme une extraction automatique du code local.

Point de précision : les classes de couleur montrent un ordre conceptuel. En CSS réel, ce sont les règles de spécificité et de cascade qui décident du résultat ; il faut vérifier le CSS final dans le navigateur, surtout lorsqu'une note combine plusieurs classes.

Fin du chapitre 05 — suite à documenter dans un chapitre distinct pour conserver une progression lisible.